Introduction to Machine Learning M146 | UCLA Spring Quarter 2026
Prof. Vwani Roychowdhury | 10th April 2026

TAs: Kartik Sharma, M. Yigit Turali

Homework #1

Due: 24th April 2026, Friday, before 11:59 pm

Problem 1 (Locally Weighted Linear Regression)

In standard linear regression, all training examples are treated equally. In many practical scenarios, however, some data points may be more important than others. To model this, we assign a positive weight $\alpha_n > 0$ to each training example.

Consider the weighted least squares objective:

$$J(w_0, w_1) = \sum_{n=1}^{N} \alpha_n \left(w_0 + w_1 x_{n,1} - y_n\right)^2 . \tag{1}$$

Here, $(x_{n,1}, y_n)$ denotes the n-th training example, and α_n controls the importance of that example.

(a) **Gradient computation.** Compute the gradient of $J(w_0, w_1)$ with respect to (w_0, w_1).

(b) **Effect of a dominant data point.** Suppose a particular training example k has weight $\alpha_k \to \infty$. What constraint does this impose on the optimal solution (w_0^*, w_1^*)?

(c) **Regularized weighted regression.** Now consider the ℓ_2-regularized objective:

$$J_\lambda(w_0, w_1) = \sum_{n=1}^{N} \alpha_n \left(w_0 + w_1 x_{n,1} - y_n\right)^2 + \lambda(w_0^2 + w_1^2), \tag{2}$$

where $\lambda > 0$.

i. Compute the gradient of $J_\lambda(w_0, w_1)$.

ii. Derive the normal equations by setting the gradients to zero in vector notation.
Hint: Define the matrix X, parameter vector w, and output vector y in the usual way. Use $A = \mathrm{diag}(\alpha_1, \ldots, \alpha_N)$.

Problem 2 (Logistic Regression and Loss Functions)

In logistic regression, we model the probability of a binary label using

$$h_{\mathbf{w}}(x) = \sigma(\mathbf{w}^T x) = \frac{1}{1 + e^{-\mathbf{w}^T x}}.$$

The predicted label is obtained by thresholding this probability at 0.5.

(a) **Behavior of the model.** Explain why logistic regression behaves like a hard (deterministic) classifier when $|\mathbf{w}^T x|$ becomes very large.

(b) **Decision boundary.** Show that the decision boundary of logistic regression is linear in x.

(c) **Effect of the loss function.**

Consider two possible loss functions:

$$J_{\text{CE}}(w) = -\sum_{i=1}^{n} \Big[y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i)) \Big],$$

$$J_{\text{MSE}}(w) = \sum_{i=1}^{n} (\sigma(w^T x_i) - y_i)^2.$$

For one training example, the gradient of cross-entropy has the form

$$(\sigma(w^T x) - y)x,$$

while squared loss introduces an additional factor involving $\sigma'(w^T x)$.

Explain qualitatively how this difference affects learning when the model is very confident but wrong.

(d) **Decision boundary and loss.** Does changing the loss from cross-entropy to squared loss change the form of the decision boundary, or only the training dynamics? Briefly explain.

Problem 3 (Ridge Regression as Augmented Least Squares)

Consider the regularized least squares problem

$$J(w) = \|Xw - y\|^2 + \lambda \|w\|^2,$$

where $X \in \mathbb{R}^{n \times d}$, $w \in \mathbb{R}^d$, $y \in \mathbb{R}^n$, and $\lambda > 0$.

(a) **Augmented least-squares form.** Show that $J(w)$ can be rewritten as

$$J(w) = \left\| \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} w - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|^2.$$

(b) **Interpretation.** Using the expression above:

- How many additional data points are effectively being added?
- What are their inputs and corresponding target values?
- Explain qualitatively how these additional data points influence the learned model.

Problem 4 (Regression on Concrete Compressive Strength Data)

Imagine you are a data scientist hired by a civil engineering firm. They design concrete mixtures for bridges, dams, and buildings, and they need a model that can predict how strong a given mixture will be *before* pouring it—saving weeks of laboratory curing time. They hand you a spreadsheet of 1030 past experiments and say: "Build us something useful."

This problem walks you through exactly what that process looks like: understanding the data, preparing it for modeling, fitting progressively more powerful models, and learning when each technique helps (and when it hurts). You may use `pandas` for loading and exploring the data and `matplotlib`/`seaborn` for plotting; all model fitting must use NumPy.

Dataset. Download the *Concrete Compressive Strength* dataset from Kaggle:

https://www.kaggle.com/datasets/maajdl/yeh-concret-data

Concrete is the most widely used construction material in the world. Its compressive strength—how much load it can bear before failing—depends on its ingredients and how long it has been cured. Each of the 1030 rows in this dataset records a laboratory mixture: the amounts of 8 components and the resulting strength in megapascals (MPa).

Feature	Name	Unit
x_1	Cement	kg/m^3
x_2	Blast Furnace Slag	kg/m^3
x_3	Fly Ash	kg/m^3
x_4	Water	kg/m^3
x_5	Superplasticizer	kg/m^3
x_6	Coarse Aggregate	kg/m^3
x_7	Fine Aggregate	kg/m^3
x_8	Age	days
y	Concrete Compressive Strength	MPa

(a) **Exploratory Data Analysis (EDA).**

The single most common mistake in machine learning is to jump straight to model fitting. Before writing a single line of regression code, you should *look at your data.* EDA helps you catch errors, spot patterns, and develop intuition that will guide every modeling decision you make later.

(i) **Basic statistics. [3 pts]** Load the dataset and print the shape, data types, and the output of `.describe()` (or equivalent). Report the number of samples, the number of features, and whether there are any missing values.
Why: This is your sanity check—you want to know how big the data is, whether columns loaded correctly, and whether there are gaps that need filling before you can do any math.

(ii) **Target distribution. [3 pts]** Plot a histogram of the target variable y (compressive strength). Is the distribution roughly symmetric, skewed, or multimodal? What are its mean and standard deviation?
Why: The shape of the target tells you what "typical" looks like. A heavily skewed target might suggest a log-transform; a multimodal one might hint at hidden sub-populations.

(iii) **Feature distributions. [4 pts]** Plot histograms for all 8 input features (a 2×4 subplot grid works well). Identify any features that are *heavily skewed* or contain a large number of zeros. Which features exhibit these properties, and why might that be the case physically?
Hint: Not every concrete mixture uses every ingredient. Fly Ash and Blast Furnace Slag are supplementary materials that are absent in many mixtures, which is why you will see a spike at zero.
Why: Features with wildly different ranges or shapes will affect standardization, and knowing which ingredients are sometimes absent helps you interpret model weights later.

(iv) **Correlations. [5 pts]** Compute the **Pearson correlation matrix** of all features and the target. Display it as a heatmap. Which feature is most positively correlated with strength? Which is most negatively correlated? Do these relationships make physical sense?
Recall: The Pearson correlation coefficient measures the *linear* relationship between two variables u and v:

$$r_{uv} = \frac{\sum_i (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_i (u_i - \bar{u})^2}\,\sqrt{\sum_i (v_i - \bar{v})^2}}.$$

It ranges from -1 (perfect negative linear relationship) to $+1$ (perfect positive linear relationship), with 0 meaning no *linear* relationship (the variables could still be related nonlinearly!). You can compute this with `np.corrcoef` or `df.corr()`.
Why: Correlations give you a quick sense of which features are likely to be important, and whether any features are highly correlated with *each other* (this is called **multicollinearity**, and it will matter when we add polynomial features later).

(v) **Pairwise scatter plots. [5 pts]** Pick the 3 features most correlated with y (in absolute value) and produce scatter plots of each against y. Do the relationships appear linear or nonlinear?
Why: If you see curved relationships here, that is a direct hint that a plain linear model will underfit—and it motivates the polynomial expansion in part (d).

(b) **Data preparation.**

You now understand the data; the next step is to get it into a form that a regression algorithm can work with. This part covers two critical concepts: **train/test splitting** (so you can honestly evaluate your model) and **feature standardization** (so the model treats all features fairly).

(i) **Train/test split. [2 pts]** Randomly split the data into 80% training and 20% test sets using `np.random.seed(42)`. Report the number of samples in each set.
Why a fixed seed: "Random" splitting means that every time you rerun your notebook, you could get a different split—and therefore different results. Setting a fixed seed makes your experiment *reproducible*: anyone who runs your code gets exactly the same numbers. This is a cornerstone of good scientific practice.
Why 42: In Douglas Adams' *The Hitchhiker's Guide to the Galaxy*, the supercomputer Deep Thought is asked for "the Answer to the Ultimate Question of Life, the Universe, and Everything." After 7.5 million years of computation, it replies: **42**. Nobody knew what the question was, but the answer stuck—and became the default random seed in machine learning. We continue the tradition.
Why hold out a test set: We hold out test data to simulate "unseen future data." If we evaluate on the same data we trained on, we have no way to detect overfitting.

(ii) **Why standardize? [3 pts]** In the raw data, Cement is measured in the hundreds of kg/m^3 while Superplasticizer is typically below 30 kg/m^3, and Age is measured in days (1–365). Explain in 2–3 sentences why fitting linear regression directly on the raw features is problematic, and how *standardization* addresses this issue.
Context: When features live on very different scales, a unit change in Cement (say, 1 kg/m^3) represents a tiny variation, while a unit change in Age (1 day) can be huge. The learned weights would need to compensate for these scale differences, making them hard to interpret and—when we get to gradient descent later—making optimization slow.

(iii) **Implement standardization. [5 pts] Standardization** (also called *z-score normalization*) transforms each feature so that it has zero mean and unit variance. For each feature j, compute the mean μ_j and standard deviation σ_j on the **training set only**, then transform:

$$\tilde{x}_{n,j} = \frac{x_{n,j} - \mu_j}{\sigma_j}.$$

After this transformation, every feature lives on roughly the same scale (centered at 0, with most values between -2 and $+2$), regardless of the original units. Apply the *same* μ_j, σ_j to the test set. Explain why it would be incorrect to compute μ_j and σ_j on the combined train+test data.
Why train-only: In a real deployment, you will not have access to future data when you build your model. Using test set statistics during preprocessing is a subtle form of **data leakage**—it lets information from the future creep into your training pipeline, making your error estimates unrealistically optimistic.

(iv) **Verify. [2 pts]** After standardization, print the mean and standard deviation of each feature on the training set (should be ≈ 0 and ≈ 1) and on the test set (will be close but not exactly 0 and 1). Why are the test set statistics not exactly 0 and 1?

(c) **Linear regression via normal equations.**

With the data explored and prepared, you are finally ready to fit your first model. We start with the simplest possible approach: ordinary least squares, solved in closed form.

Implement OLS by solving the normal equations:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

where $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ includes a column of ones for the bias term.

(i) **[3 pts]** Report the training and test **Mean Squared Error** (MSE).
Recall: The MSE measures the average squared difference between predictions and true values:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2.$$

A lower MSE means better predictions. Training MSE tells you how well the model fits the data it was trained on; test MSE tells you how well it *generalizes* to new data.

(ii) **[4 pts]** Which feature has the largest (absolute) weight? Give an intuitive explanation for why this makes sense physically.
Note: Because you standardized the features, the weights are directly comparable—a larger weight genuinely means "this feature matters more," which would *not* be true on the raw data. This is another reason we standardize.

(d) **Polynomial feature expansion.**

If your scatter plots from part (a)(v) showed curved relationships, then a linear model is fundamentally limited: it can only draw flat hyperplanes through the data. The key insight from class is that we do not need a new algorithm—we can make linear regression capture nonlinear patterns by transforming the inputs first.

Instead of fitting $y = \mathbf{w}^\top \mathbf{x}$, we fit

$$y = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}),$$

where $\boldsymbol{\phi}(\mathbf{x})$ maps the original features to a higher-dimensional space. For a degree-p polynomial expansion with d features, we include all monomials up to degree p:

$$\boldsymbol{\phi}(\mathbf{x}) = \left[1,\ x_1,\ x_2,\ \ldots,\ x_d,\ x_1^2,\ x_1x_2,\ \ldots,\ x_d^p\right]^\top.$$

The model is still "linear in the parameters" $\mathbf{w}$, so the normal equations still apply—we just feed in a wider matrix.

(i) **[3 pts]** How many features does $\boldsymbol{\phi}(\mathbf{x})$ produce for $d = 8$ and $p = 2$? What about $p = 3$?
Hint: The number of monomials of degree $\leq p$ in d variables is $\binom{d+p}{p}$.

(ii) **[5 pts]** Construct the degree-2 polynomial feature matrix and fit linear regression on the expanded features using the normal equations. Report the training and test MSE. How does it compare to part (c)?
What to expect: If the true relationship between ingredients and strength is nonlinear (and it is—e.g., too much water weakens concrete), then adding squared terms and interactions should help.

(iii) **[5 pts]** Now try degree $p = 3$. What happens to the training MSE versus the test MSE? Explain.
What to watch for: You will likely see the training MSE drop further, but the test MSE may *increase*. This is the hallmark of **overfitting**: the model memorizes training noise rather than learning the true pattern. With $\binom{11}{3} = 165$ features and only ~800 training samples, the model has too many knobs to turn relative to the amount of data.

(e) **Ridge regression on polynomial features.**

Overfitting happens when the model is too flexible for the amount of data. One principled fix is **regularization**: we add a penalty that discourages large weights, effectively telling the model "you can use polynomial features, but do not rely on any single one too heavily." Ridge regression does this by modifying the normal equations:

$$\mathbf{w}_\lambda^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

The hyperparameter $\lambda > 0$ controls how much we penalize large weights. When $\lambda = 0$ we recover OLS; as $\lambda \to \infty$, the weights shrink toward zero and the model becomes very simple (high bias, low variance).

Using the degree-3 polynomial features from part (d)(iii):

(i) **[3 pts]** Sweep λ over the values $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$. For each λ, report training and test MSE.

(ii) **[5 pts]** Plot training MSE and test MSE (on the same axes) as a function of $\log_{10}(\lambda)$.
What to look for: You should see the classic U-shaped test error curve. Too little regularization $\to$ overfitting; too much $\to$ underfitting. The sweet spot is in between.

(iii) **[5 pts]** At what value of λ is the test MSE minimized? Compare this to the unregularized degree-3 result from part (d)(iii) and the linear result from part (c). What do you conclude about the bias-variance trade-off?

(iv) **[5 pts]** Plot $\|\mathbf{w}_\lambda^*\|_2$ as a function of $\log_{10}(\lambda)$. Relate this to the augmented least-squares interpretation from Problem 3 of this homework: regularization is equivalent to adding "fake" data points that pull the weights toward zero.

(f) **Cross-validation for model selection.**

In part (e), you selected the ridge parameter λ by comparing performance on a single train/test split. This is a useful first step, but the quality of that choice can depend heavily on the particular random split you happened to get. A different split might produce a different "best" value of λ. To reduce this dependence, we use **K-fold cross-validation**.

In **K-fold cross-validation**, we divide the available training data into K approximately equal-sized parts called *folds*. Then we repeat the following process K times:

- choose one fold to act as a validation set,
- train the model on the remaining $K-1$ folds,
- evaluate the model on the held-out fold.

Each fold gets used once as the validation set. We then average the validation errors across all K runs. This gives a more stable estimate of generalization performance than relying on a single split.

For this problem, use **5-fold cross-validation** on the **training portion only** of the data.

(i) **Cross-validation for ridge regression. [6 pts]** Using the degree-3 polynomial features from part (d)(iii), perform **5-fold cross-validation** on the training data to choose the best ridge parameter

$$\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}.$$

For each value of λ, report the **mean validation MSE** across the 5 folds.

(ii) **Best hyperparameter and final test result. [4 pts]** Which value of λ gives the lowest average validation MSE? Retrain ridge regression on the **full training set** using this value, and then report the final **test MSE**. Compare this test MSE to the one obtained in part (e) using a single split. Are the conclusions similar?

(iii) **Why not use the test set? [3 pts]** Explain why the test set must not be used inside the cross-validation procedure. What kind of problem would occur if you repeatedly used the test set to choose λ?

(g) **Weighted least squares.**

So far, every training example has been treated equally. But what if some examples are more important than others? In practice, this happens when data collection is imbalanced: the engineering firm has tested thousands of 28-day-old samples (the industry standard curing time), but only a handful of samples at other ages. A plain OLS model will naturally fit the 28-day region well—because that is where most of the data lives—and may perform poorly on early-age or late-age predictions.

The `Age` feature (x_8) illustrates this perfectly.

(i) **[3 pts]** Plot a histogram of the `Age` feature. Verify that the distribution is heavily skewed. How many distinct age values appear in the dataset, and what fraction of samples have Age = 28?

(ii) **[7 pts]** To force the model to pay more attention to underrepresented ages, define the sample weight as the inverse frequency of each age group:

$$\alpha_n = \frac{N}{|\{i : x_{i,8} = x_{n,8}\}|},$$

so that rare ages receive larger weights and common ages receive smaller weights. Using the original (degree-1) features, solve the *weighted* normal equations from Problem 1 of this homework:

$$\mathbf{w}_\alpha^* = (\mathbf{X}^\top \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{A} \mathbf{y},$$

where $\mathbf{A} = \mathrm{diag}(\alpha_1, \ldots, \alpha_N)$.

Report training and test MSE (unweighted) and compare to part (c). Does weighting help or hurt overall MSE? Why might a practitioner still prefer this model?

Reflection: Overall MSE is dominated by the majority group (28 days). A model optimized for overall MSE might systematically fail on early-age concrete, which could be dangerous in time-critical construction. Weighted regression trades a bit of average accuracy for better fairness across the full age range.

(h) **Summary and discussion. [5 pts]**

Create a single table summarizing all models you have trained (linear, degree-2, degree-3, ridge with best λ, weighted linear) and their training/test MSE. In 3–5 sentences, discuss:

- Which model would you recommend to the engineering firm and why?
- What is the role of regularization in the polynomial setting?
- When might weighted regression be preferred despite not achieving the lowest test MSE?

Problem 5 (Logistic Regression on the Spaceship Titanic Dataset)

It is the year 2912. The interstellar passenger liner *Spaceship Titanic*, carrying nearly 13,000 passengers on its maiden voyage to three newly habitable exoplanets, has collided with a space-time anomaly hidden within a dust cloud. Almost half of the passengers were transported to an alternate dimension. Rescue teams have recovered the ship's damaged computer system, which contains partial records for about 8,700 passengers. Your job: build a classifier that predicts which passengers were transported, so the rescue teams know who to search for in this dimension and who to search for in the other.

Unlike the concrete dataset, this data is **messy**—there are missing values, a mix of numerical and categorical features, and columns that look informative but are actually useless. A major goal of this problem is to teach you how to handle this messiness, because real-world data almost always looks like this.

You may use `pandas` for loading, exploring, and preprocessing the data and `matplotlib`/`seaborn` for plotting; all model fitting and gradient computation must use NumPy.

Dataset. Download the *Spaceship Titanic* dataset from Kaggle:

https://www.kaggle.com/competitions/spaceship-titanic/data

Kaggle provides two files: `train.csv` ($N \approx 8700$ passengers *with* the `Transported` label) and `test.csv` (≈ 4300 passengers *without* labels, used for Kaggle submissions). Since we cannot

evaluate on unlabeled data, we will work exclusively with `train.csv` and create our own internal splits.

The training file contains the following columns:

Column	Type	Description
`PassengerId`	ID	Unique identifier (`gggg_pp`: travel group and person number)
`HomePlanet`	Categorical	Planet of departure (Earth, Europa, Mars)
`CryoSleep`	Boolean	Whether the passenger elected to be in cryogenic sleep for the voyage
`Cabin`	Categorical	Cabin number in the format `deck/num/side`
`Destination`	Categorical	Destination planet
`Age`	Numerical	Age of the passenger
`VIP`	Boolean	Whether the passenger paid for VIP service
`RoomService`	Numerical	Amount billed at room service
`FoodCourt`	Numerical	Amount billed at the food court
`ShoppingMall`	Numerical	Amount billed at the shopping mall
`Spa`	Numerical	Amount billed at the spa
`VRDeck`	Numerical	Amount billed at the VR deck
`Name`	String	First and last name of the passenger
`Transported`	Boolean	**Target:** was the passenger transported to the alternate dimension?

(a) **Exploratory Data Analysis (EDA).**

The rescue team has handed you a CSV file and said "figure out what is in here." Before you can build any model, you need to understand the data: What does it look like? What is broken? What patterns might be useful? This is EDA, and skipping it is the fastest way to build a model that looks great on paper and fails in practice.

(i) **First look. [2 pts]** Load the training CSV. Print the shape, column names, and data types. Display the first 5 rows. How many features are numerical? How many are categorical?
Why: You need to know what you are working with before anything else. Data types determine which preprocessing steps are needed: you cannot take the mean of a categorical column, and you cannot one-hot encode a number.

(ii) **Missing values. [3 pts]** For each column, report the number and percentage of missing values. Which columns have the most missing data? Are the missing values concentrated in a few columns or spread throughout? Display your results as a table or bar chart.
Why: Missing data is the most common problem in real-world datasets. If you ignore it, NumPy will propagate `NaN`s through all your computations and every gradient update will produce garbage. You need to know *where* the gaps are before deciding *how* to fill them.

(iii) **Target balance. [2 pts]** What fraction of passengers were transported? Is the dataset roughly balanced or significantly imbalanced?
Why: In a heavily imbalanced dataset (say, 95% negative), a model that always predicts "not transported" would achieve 95% accuracy while being completely useless.

Knowing the balance tells you whether **accuracy** (the fraction of correct predictions) is a meaningful metric or whether you need to be more careful.

(iv) **Numerical feature distributions.** [**4 pts**] Plot histograms of all numerical features (`Age`, `RoomService`, `FoodCourt`, `ShoppingMall`, `Spa`, `VRDeck`). Which features are heavily right-skewed? What does it mean that the vast majority of passengers have spending values near zero?
Why: Skewed features can dominate a model in unexpected ways. If most spending values are 0 but a few are in the thousands, the model might focus all its attention on those outliers. Seeing this now will motivate the log-transform and zero-spending indicator in part (e).

(v) **Categorical feature distributions.** [**4 pts**] For `HomePlanet`, `Destination`, `CryoSleep`, and `VIP`, plot bar charts showing the count of each category. For each, also show the *transportation rate* (fraction transported) per category. Which categorical feature appears most predictive of the target?
Why: If the transportation rate varies dramatically across categories (e.g., 80% for CryoSleep=True vs. 30% for CryoSleep=False), that feature is likely informative. This gives you an early sense of which features the model should rely on—and lets you sanity-check the model later.

(vi) **Spending vs. target.** [**5 pts**] Create box plots or violin plots of each spending feature split by `Transported` (0 vs. 1). Do transported passengers tend to spend more or less? Propose a hypothesis for why this pattern exists.
Hint: Think about what happens when a passenger is in cryogenic sleep—can they visit the food court?

(b) **Data cleaning and preprocessing.**

You have looked at the data and found missing values, categorical columns, and features on wildly different scales. None of these are compatible with the linear algebra you will use for logistic regression. This part turns messy real-world data into a clean numerical matrix $\mathbf{X}$ that you can actually multiply by a weight vector.

Every decision in this section has a "why"—preprocessing is not arbitrary busywork; each step exists to prevent a specific failure mode.

(i) **Dropping uninformative columns.** [**2 pts**] Remove `PassengerId`, `Name`, and `Cabin` from the dataset. Briefly justify why each is not directly useful for logistic regression.
Why we drop: `PassengerId` is a unique identifier with no predictive content (knowing someone is passenger #4217 tells you nothing about whether they were transported). `Name` is a string that would require NLP to use. `Cabin` has exploitable structure (`deck/num/side`), but parsing it is beyond the scope of this problem.
General lesson: Not every column in a dataset belongs in your model. Keeping irrelevant features adds noise and slows training without improving predictions.

(ii) **Handling missing values—categorical features.** [**3 pts**] For `HomePlanet`, `Destination`, `CryoSleep`, and `VIP`, fill missing values with the **mode** (most frequent value) computed on the training set.
Recall: The **mode** of a dataset is the value that appears most often. For categorical data, it is the only standard measure of central tendency that makes sense—unlike the mean or median, it does not require numerical ordering.
Why the mode: Categorical features are labels like "Earth" or "Mars"—you cannot average them. The mode is the simplest reasonable guess: if 60% of passengers departed from Earth, then a passenger with unknown origin is most likely from Earth.

(iii) **Handling missing values—numerical features. [3 pts]** For `Age` and the 5 spending features, fill missing values with the **mean** computed on the training set.
Why the mean: Filling with zero would be wrong—it would imply the passenger is a newborn (Age=0) or deliberately spent nothing. The mean says "we do not know, so we guess the average passenger." It is not perfect (it ignores correlations with other features), but it is a principled default.
Why training set only: Just as with standardization, imputation statistics must come from the training set. In deployment, you will not have access to future data when you fill in a missing field. Using test data for imputation is **data leakage**.

(iv) **Encoding categorical variables. [5 pts]** Logistic regression requires numerical inputs: you cannot multiply a weight by the string "Europa." Convert `CryoSleep` and `VIP` to binary (0/1). For `HomePlanet` (3 categories) and `Destination` (3 categories), apply **one-hot encoding**.
Recall: **One-hot encoding** replaces a categorical column with k binary indicator columns, one per category. For example, `HomePlanet` $\in$ {Earth, Europa, Mars} becomes three columns: `is_Earth`, `is_Europa`, `is_Mars`, where exactly one is 1 and the rest are 0 for each row. We drop one of the k columns (using $k-1$ indicators) to avoid **multicollinearity**.
Why $k-1$ instead of k: If you keep all k indicator columns, they always sum to 1, which means one column is a perfect linear combination of the others. This makes $\mathbf{X}^\top\mathbf{X}$ singular (non-invertible). Dropping one category (the "reference" category) breaks this dependency.
How many total features do you have after encoding? Print the first 5 rows of the processed feature matrix to verify everything looks correct.

(v) **Train / validation / test split. [4 pts]** In Problem 4 we split data into two sets: train and test. Here we introduce a third: the **validation set**.

Using `np.random.seed(42)` (see Problem 4 for why 42), split `train.csv` into three disjoint subsets:

- **Training set** (60%) — used to fit model parameters $\mathbf{w}$.
- **Validation set** (20%) — used to tune hyperparameters (like λ) and compare model variants.
- **Test set** (20%) — touched *only once*, at the very end, to report final performance.

Report the number of samples in each set, and verify that the class balance (fraction transported) is approximately preserved across all three.
Why three sets instead of two? In Problem 4, we used the test set to compare models (e.g., picking the best λ). Strictly speaking, this is cheating: every time you peek at test performance and then change your model, the test set is no longer "unseen." The validation set solves this problem—you can look at validation accuracy as many times as you like while developing your model, and the test set stays truly untouched until the final evaluation. This is how machine learning is done in practice: **train** on training data, **select** your best model using validation data, and **report** final numbers on test data.
Hint: One clean way to do this is to first split into 80%/20% (train+val vs. test), then split the 80% into 75%/25% (train vs. val), giving you a 60/20/20 ratio overall.

(vi) **Feature standardization. [3 pts]** Standardize all features to zero mean and unit variance using the **training set** statistics only (see Problem 4(b) for details). Apply the same transformation to both the validation set and the test set.
Why this matters for gradient descent: Unlike the normal equations (which give the exact solution in one step), **gradient descent** takes iterative steps downhill on the

loss surface. When features have very different scales, the loss surface becomes an elongated "valley"—the gradient points mostly in the direction of the large-scale features, forcing you to use a tiny learning rate to avoid overshooting. Standardization makes the surface more circular, so gradient descent converges much faster.

(c) **Logistic regression with cross-entropy loss.**

The data is clean. Now you can build a classifier. Logistic regression models the probability of transportation as $\sigma(\mathbf{w}^\top \mathbf{x})$, where σ is the sigmoid function. But which loss function should we use to train it? In Problem 2(c) of this homework you analyzed two options theoretically; now you will see the difference empirically.

Implement **gradient descent** to minimize the cross-entropy (CE) loss:

$$J_{\mathrm{CE}}(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\big[y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1-y_i)\log(1-\sigma(\mathbf{w}^\top \mathbf{x}_i))\big],$$

where $\sigma(z) = 1/(1+e^{-z})$.

Recall: **Gradient descent** is an iterative optimization algorithm. Starting from an initial guess $\mathbf{w}^{(0)}$ (e.g., a vector of zeros), you repeatedly update:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\, \nabla_{\mathbf{w}} J(\mathbf{w}^{(t)}),$$

where $\eta > 0$ is the *learning rate* (step size). Each step moves $\mathbf{w}$ in the direction that most steeply decreases the loss.

(i) [**3 pts**] Write down the gradient $\nabla_{\mathbf{w}} J_{\mathrm{CE}}$ and verify it matches the form given in Problem 2(c) of this homework.

(ii) [**5 pts**] Run gradient descent with learning rate $\eta = 0.1$ for 1000 iterations. Initialize $\mathbf{w}^{(0)} = \mathbf{0}$. Plot the training loss J_{CE} as a function of iteration number. Does the loss converge?

(iii) [**3 pts**] Report training and **validation** accuracy (fraction of correctly classified examples using a threshold of 0.5).

(d) **Logistic regression with squared loss.**

In Problem 2(c), you showed that the MSE gradient picks up an extra factor of $\sigma'(\mathbf{w}^\top \mathbf{x})$, which vanishes when the sigmoid is saturated. Let us see what that means in practice.

Train the same model using the mean squared error loss:

$$J_{\mathrm{MSE}}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\big(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i\big)^2.$$

(i) [**4 pts**] Derive the gradient $\nabla_{\mathbf{w}} J_{\mathrm{MSE}}$. Identify the extra multiplicative factor compared to the cross-entropy gradient.

(ii) [**5 pts**] Using the same learning rate $\eta = 0.1$ and 1000 iterations, plot the MSE training loss curve *alongside* the CE training loss curve from part (c)(ii) (use two separate y-axes or normalize appropriately so both are visible).
What to watch: The CE loss should drop smoothly. The MSE loss may stall or converge much more slowly—this is the **vanishing-gradient effect** you predicted theoretically.

(iii) **[4 pts]** Report training and validation accuracy. Is performance better, worse, or similar to cross-entropy? Relate your findings to your answer in Problem 2(c): when the model is confidently wrong (e.g., predicts 0.99 for a true label of 0), the CE gradient pushes hard to correct the mistake, while the MSE gradient is multiplied by $\sigma'(\cdot) \approx 0$ and barely moves.

(e) **Feature engineering via input mapping.**

Your EDA in part (a) revealed that spending features are heavily right-skewed and that cryosleep passengers spend nothing. Raw spending values may not be the best representation—perhaps what matters is not *how much* someone spent, but *whether* they spent at all, or the *order of magnitude* of their spending. This is the idea behind feature engineering: transforming inputs to make the signal easier for a linear model to find.

(i) **[5 pts]** Create two new features: (1) a binary indicator $z_i = \mathbf{1}[\text{total spending}_i = 0]$ for whether the passenger spent nothing at all, and (2) $\log(1+\text{total spending}_i)$, which compresses the long tail of high spenders into a more manageable range. Add both to your feature set (standardize the new continuous feature). Retrain the CE-loss model. Does validation accuracy improve?
Why $\log(1+\cdot)$*:* The $+1$ avoids $\log(0)$. The log-transform turns multiplicative differences into additive ones: the distinction between spending \$0 vs. \$10 is captured as much as \$1000 vs. \$10,000, which is often more meaningful.

(ii) **[5 pts] Polynomial input mapping.** Construct degree-2 **interaction features** *among the 5 spending variables only* (i.e., $x_j x_k$ for all pairs $j \leq k$).
Recall: An **interaction feature** is the product of two existing features. It allows a linear model to capture relationships that depend on combinations of variables—for example, "passengers who spent at *both* the Spa and VRDeck are more likely to be transported" cannot be expressed as a linear function of Spa and VRDeck alone, but *can* be captured by including the product Spa $\times$ VRDeck.
How many new features does this add? Retrain and report validation accuracy.

(iii) **[3 pts]** Does the polynomial expansion help significantly here? Discuss whether the decision boundary for this problem is likely linear or nonlinear, and why.

(f) **ℓ_2-regularized logistic regression.**

Adding polynomial features gives the model more expressive power, but also more opportunity to overfit—especially the interaction terms, which can latch onto noise. Regularization reins in this complexity by penalizing large weights.

Add an ℓ_2 penalty to the cross-entropy loss:

$$J_\lambda(\mathbf{w}) = J_{\text{CE}}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2.$$

Using the expanded feature set from part (e)(ii):

(i) **[3 pts]** Derive the gradient $\nabla_{\mathbf{w}} J_\lambda$.
Note: This is analogous to ridge regression from Problem 4—the penalty adds $\lambda\mathbf{w}$ to the gradient, gently pushing all weights toward zero.

(ii) **[4 pts]** Sweep $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. For each value, report training and **validation** accuracy.

(iii) **[5 pts]** Plot training and validation accuracy as a function of $\log_{10}(\lambda)$. At which λ is the **validation** accuracy maximized? This is your best hyperparameter choice. How

does regularization interact with the polynomial features?
What to expect: Without regularization, the interaction features may overfit slightly. A moderate λ should improve validation accuracy by suppressing noisy interaction weights while keeping the genuinely useful ones.
Why validation, not test: This is exactly why we created a validation set. You are making a *decision* (which λ to use) based on this data, so it is no longer "unseen." The test set must remain untouched until the final evaluation in part (g).

(g) **Cross-validation for hyperparameter tuning.**

In part (f), you selected the regularization strength λ using a single validation set. This is a standard approach, but the selected value can depend on the particular split of the data. To make hyperparameter selection more stable, we can use **K-fold cross-validation** on the training data.

Recall that in **K-fold cross-validation**, the training data is split into K folds. For each fold:

- train the model on the other $K-1$ folds,
- evaluate it on the held-out fold,
- record the validation performance.

At the end, we average the validation performance over all K folds and use this average to compare hyperparameter choices.

In this problem, use **5-fold cross-validation** on the **training set only**. Do not use the test set until the very end.

(i) **Cross-validation sweep for regularization. [6 pts]** Using the expanded feature set from part (e)(ii), perform **5-fold cross-validation** on the training data to choose the best regularization parameter

$$\lambda \in \{0,\ 10^{-4},\ 10^{-3},\ 10^{-2},\ 10^{-1},\ 1\}.$$

For each λ, report the **mean validation accuracy** across the 5 folds.

(ii) **Compare with single-split tuning. [4 pts]** Compare the best λ chosen by cross-validation with the best λ chosen in part (f) using a single validation split. Do they match? If not, give one possible reason why the two procedures may lead to different choices.

(iii) **Final model evaluation. [4 pts]** Retrain the model on the **full training set** using the value of λ selected by cross-validation, and report the final **test accuracy**. Compare this to the test accuracy of the model selected using only a single validation split.

(iv) **Why is the test set used only once? [3 pts]** Briefly explain why the test set should be touched only once at the very end. What does it mean for the test set to provide an "honest" estimate of generalization?

(h) **Analysis and discussion.**

Now—and *only* now—evaluate your best model on the **test set**. This is the first time you are touching test data, so this number is an honest estimate of how well your model generalizes.

(i) **[3 pts]** Take the best model from your experiments (best feature set + best λ). Report its **test accuracy**. Is it close to the validation accuracy, or is there a significant

gap?
Why this matters: If test accuracy is much worse than validation accuracy, your model may have overfit to the validation set through repeated tuning. If they are close, your validation procedure was sound.

(ii) [**3 pts**] Examine the learned weight vector from your best model. Which 3 features have the largest (absolute) weights? Do these make intuitive sense given the problem setting and the patterns you discovered in your EDA?

(iii) [**5 pts**] The **decision boundary** of logistic regression is the set $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} = 0\}$—this is the hyperplane where the model is exactly 50/50 on its prediction. Pick two informative features and produce a 2D scatter plot of the test data colored by true label, with the learned decision boundary overlaid. Comment on the quality of the linear separator.
Why this matters: This is the geometric view of Problem 2(b) from your homework—the decision boundary is always a hyperplane (or a line in 2D), no matter how many features you have.

(iv) [**5 pts**] Summarize your results in a table (model variant, number of features, train/validation/test accuracy). In 3–5 sentences, discuss the relative importance of (i) loss function choice, (ii) feature engineering, and (iii) regularization for this task. Which mattered most? Which mattered least? Would you recommend cross-entropy or MSE loss to someone building a classifier?